PRD — Mr. White / Impostor Web (Mobile-first)

Versión 1.1 (MVP) — En vivo + Remoto (sin audio/video)

Fecha: 21 de febrero de 2026
Autor: (completar)
Owner del producto: (vos)

Decisiones ya definidas (para evitar ambigüedades)

Tema	Decisión	Impacto en MVP
Audio/Video	No incluido en MVP. Remoto usa llamada externa.	Reduce complejidad; UI solo gestiona estado del juego.
Undercover	Opcional (puede ser 0).	Mismo motor soporta 1 o 2 palabras.
Revelación al eliminar	Solo se revela el rol (no la palabra).	Más difícil y más divertido; evita leaks.

Contenido (resumen)

- 1. Visión y objetivos
- 2. Alcance (MVP) y fuera de alcance
- 3. Personas y casos de uso
- 4. Definición del juego (roles, fases, victoria)
- 5. Requisitos funcionales (MUST/SHOULD/COULD)
- 6. UX anti-spoiler (modo EN VIVO)
- 7. Arquitectura técnica (server local + realtime)
- 8. Contratos de eventos (Socket) y modelo de datos
- 9. Especificación de implementación (Codex-ready)
- 10. Plan de pruebas y criterios de aceptación
- 11. Deployment gratis en Mac (túneles)
- 12. Roadmap (LiveKit A/V)
- Apéndices (mazos de palabras, comandos, referencias)

1. Visión y objetivos

Construir una versión web, mobile-first, del juego social Mr. White / Impostor que se comparte por QR o link. Debe soportar dos modos (EN VIVO y REMOTO), distribuir roles y palabras sin papel y sincronizar turnos y votos en tiempo real, manteniendo el foco en la interacción humana (hablar, mirar, deducir).

Objetivos (Goals)

- Crear una sala y compartir por QR/link para que cualquiera se una desde el celular (sin instalar apps).
- Dos modos: EN VIVO (anti-spoiler, teléfonos visibles) y REMOTO (sincronizado; A/V externo).
- Operar desde una Mac hosteando localmente, expuesto gratis con un túnel (Cloudflare Quick Tunnels recomendado).
- Juego completo: lobby -> reparto -> turnos -> votación -> eliminación -> victoria.
- Privacidad y fricción mínima: sin cuentas, sin emails, sin SaaS pago.

Métricas de éxito (MVP)

- Tiempo desde crear sala hasta empezar partida: < 60 segundos con 6-10 jugadores.
- 0 leaks accidentales por UI en modo EN VIVO (pantalla neutra por defecto; secreto efímero; bloqueo post-reveal).
- Reconexión: si un jugador recarga, vuelve al estado correcto sin romper la partida.
- Soporta 10-15 jugadores en una sala con latencia percibida aceptable (< 300 ms para updates).

2. Alcance (MVP) y fuera de alcance

Incluido en MVP

- Web app mobile-first (iOS Safari / Android Chrome) + pantalla pública opcional.
- Sala con código corto + QR en pantalla.
- Asignación de roles: Civil, Undercover (opcional), Mr White.
- Selección de palabras: random desde mazo local o custom A/B.
- Fases: Lobby, Reveal, Clues (turnos), Vote, Resolve, End.
- Votación secreta + resolución con desempate aleatorio entre empatados.
- Revelación al eliminar: solo rol.
- Revelación final: mostrar palabras A/B al terminar la partida (por defecto).

Fuera de alcance (no-goals)

- Audio/video embebido (se usa llamada externa).
- Cuentas, login, perfiles, historial persistente o analytics cloud.
- Anti-cheat perfecto (capturas, grabación de pantalla).
- Matchmaking público o discovery de salas.
- Multijuego simultáneo con escalado cloud (MVP es local en tu Mac).

3. Personas y casos de uso

Personas

Persona	Objetivo principal	Notas
Host	Crea la sala, configura roles/palabras, inicia fases, puede cerrar votación y resetear sala.	Controla el flujo; requiere hostKey.
Jugador	Se une por link/QR, ve su rol/palabra de forma privada, habla pistas, vota, sigue el estado.	UX anti-spoiler en EN VIVO.
Pantalla pública (opcional)	Dispositivo visible para todos mostrando solo info pública (turno, timer, fase).	Se conecta como spectator.

Casos de uso clave

- Presencial: QR en la mesa, todos entran, reciben rol/palabra sin que otros lo vean, y juegan hablando.
- Remoto: link por chat, todos entran desde su casa y se coordinan con llamada externa; la web maneja turnos y votos.
- Demo / streaming: usar pantalla pública para ver progreso sin secretos.

4. Definición del juego

Roles

- Civil: recibe Palabra A.
- Undercover (opcional): recibe Palabra B (similar).
- Mr White: no recibe palabra.

Fases (state machine)

- Lobby: jugadores se unen y marcan Listo.
- Reveal: única fase donde se puede ver el secreto (obligatorio bloquear re-ver en modo EN VIVO).
- Clues: turnos para decir 1 pista hablada (timer opcional).
- Vote: cada jugador vivo vota a quién eliminar.
- Resolve: se elimina a alguien, se revela solo el rol, se evalúa victoria.
- End: se anuncia ganador y se revelan palabras A/B (por defecto).

Reglas de victoria (presets MVP)

Preset SIMPLE (recomendado)

- Civiles ganan si Mr White es eliminado.

- Impostores ganan si Mr White llega a los últimos 2 jugadores vivos.
- Undercover (si existe) gana junto con impostores.

Preset CLASSIC_GUESS (opcional)

- Igual que SIMPLE, pero si Mr White es eliminado tiene 1 oportunidad de adivinar Palabra A para ganar.
- Si falla, ganan civiles.

Desempate en votación (MVP)

Si hay empate por mayor cantidad de votos, el servidor elimina al azar a uno de los empatados.

5. Requisitos funcionales

5.1 MUST (imprescindibles)

Sala

- Crear sala con roomCode corto + URL de acceso.
- Generar QR en la web para compartir.
- Join sin login: nombre único dentro de sala.

Lobby

- Lista de jugadores + estado de listo.
- Host puede cerrar ingreso al iniciar.

Configuración (host)

- Elegir modo: EN VIVO o REMOTO.
- Configurar conteo de roles con validación (suma = # jugadores listos).
- Elegir palabras: random desde mazo local o custom A/B.
- Configurar timer (segundos) o manual.

Secreto

- Asignación de rol/palabra por servidor y entrega privada al socket del jugador.
- UI mantener presionado para revelar con auto-ocultado.
- En EN VIVO: secreto visible solo durante Reveal; bloqueo total post-Reveal.

Turnos

- Orden aleatorio por ronda entre vivos.
- Timer server-authoritative si está habilitado.
- Indicar claramente quién habla ahora.

Votación

- Voto secreto; una sola vez por jugador vivo (permitir cambiar hasta cierre es OK).

- Cierre automático al votar todos o manual por host.
- Resolver empate por eliminación aleatoria entre empatados.

Eliminación y fin

- Eliminar jugador: pasa a muerto y no vota/participa.
- Revelar solo el rol del eliminado.
- Evaluar victoria tras cada eliminación.
- Pantalla final: ganador + revelar palabras A/B.

5.2 SHOULD (muy deseables)

- Pantalla pública (ruta /public) con fase, turno, timer, vivos/eliminados y progreso de votación (sin secretos).
- Reconexión: token de jugador en localStorage para re-asociar al recargar.
- Host puede expulsar jugadores antes de iniciar.
- Beep opcional al terminar turno.
- Botón Reset sala al finalizar (limpia estado).

5.3 COULD (nice to have)

- PWA (Add to Home Screen).
- Multi-idioma ES/EN.
- Export de logs (JSON) por ronda para debugging.
- Web Speech API para avisos (sin transcripción).

6. UX anti-spoiler (modo EN VIVO)

Principio: los teléfonos pueden quedar a la vista sin revelar información que afecte el juego. La interacción principal es hablada. La pantalla debe estorbar lo mínimo.

Reglas UX (EN VIVO)

- Pantalla neutra por defecto (segura si alguien mira).
- Revelación por gesto intencional (press-and-hold 2-3s).
- Auto-ocultar al soltar o al cumplir el tiempo.
- Bloquear re-ver palabras tras cerrar la fase Reveal (MUST).
- Mismo layout neutro para todos los roles (no delatar por UI).
- Evitar texto persistente tipo “Sos Mr White” fuera de Reveal.

Pantallas (resumen)

Pantalla	Qué muestra
Home	Crear sala / Unirse por código.
Lobby	Lista jugadores + listo; host configura.
Reveal	Tarjeta secreta (press-and-hold).

Clues	Turno actual + timer; UI mínima.
Vote	Lista de vivos para votar; confirmación.
Resolve	Eliminado + rol revelado; siguiente ronda.
End	Ganador + revelar palabras A/B.

7. Arquitectura técnica

Stack recomendado (Codex-friendly)

- Server: Node.js + TypeScript + Socket.IO (realtime) + Express para servir assets (opcional).
- Client: Vite + React + TypeScript.
- Shared: tipos TypeScript compartidos (Room, Player, eventos).
- Persistencia: memoria del proceso (MVP). Reiniciar server = resetear salas.

Topología

Tu Mac corre el server. Los clientes acceden vía LAN (presencial) o vía túnel HTTPS (remoto). El server es autoritativo para el estado del juego y el timer.

Requisitos no funcionales

- Soportar 10-15 jugadores en una sala.
- Latencia: updates de turnos y votos deben sentirse instantáneos.
- Compatibilidad: Safari iOS y Chrome Android.
- Privacidad: no almacenar PII ni logs persistentes por defecto.
- Seguridad básica: roomCode aleatorio, hostKey para acciones de host, rate-limit de joins.

8. Contratos realtime y modelo de datos

8.1 Eventos Socket (contratos)

Client -> Server

- room:create { mode }
- room:join { roomCode, name, playerToken? }
- room:ready { ready: true }
- host:configure { settings, wordSource }
- host:start {}
- host:closeReveal {}
- turn:next {} (si manual)
- vote:cast { targetPlayerId }
- vote:close {}
- mrwhite:guess { guess } (si aplica)
- host:resetRoom {}

Server -> Client

- room:state_public { roomPublicState }
- player:secret { role, wordOrNull, revealAllowed } (solo a ese jugador)
- phase:update { status }
- turn:update { currentSpeakerId, timeRemaining }

- vote:update { votesCast, votesTotal }
- resolve:elimination { eliminatedPlayerId, revealedRole }
- game:end { winner, reason, wordPair }

8.2 Modelo de datos (MVP)

Room (campos principales)

code, mode, status, createdAt, settings, wordPair, players, alivePlayerIds, roundNumber, turnOrder, currentTurnIndex, votes, lastElimination

Player (campos principales)

id, name, isHost, connected, role, assignedWord, ready, socketId

8.3 Estado público vs secreto

El server debe distinguir entre (a) estado público y (b) estado secreto por jugador. El secreto se entrega por evento privado al socket del jugador y no se incluye en room:state_public.

9. Especificación de implementación (Codex-ready)

9.1 Estructura de repositorio (sugerida)

```
repo/
  client/
    src/
      pages/
        Home.tsx
        Lobby.tsx
        Game.tsx
        Public.tsx
      components/
        QRCode.tsx
        SecretCard.tsx
        TurnBanner.tsx
        VotePanel.tsx
      state/
        socket.ts
        roomStore.ts
      assets/words_es.json
    vite.config.ts
  server/
    src/
      index.ts
      rooms/
        roomStore.ts
        roomLogic.ts
        transitions.ts
      socket/
        handlers.ts
        auth.ts
  shared/
    types/
      room.ts
      events.ts
      words.ts
  README.md
```

9.2 Contratos precisos (TypeScript)

Codex debe implementar estos tipos en shared/types/events.ts y re-usarlos en client y server.

```
Role = 'CIVIL' | 'UNDERCOVER' | 'MR_WHITE'
```

Mode = 'LIVE' | 'REMOTE'
Status = 'LOBBY' | 'REVEAL' | 'CLUES' | 'VOTE' | 'RESOLVE' | 'END'
Settings = { roleCounts: {civil:number; undercover:number; mrWhite:number}; turnSeconds:number|null; allowSecretReviewInRemote:boolean; mrWhiteCanGuessOnElim:boolean; showVoteTally:boolean; winPreset:'SIMPLE'|'CLASSIC_GUESS' }
WordPair = { a: string; b: string; category?: string }
RoomPublicState = { code; mode; status; playersPublic:[{id;name;ready;alive;isHost}]; roundNumber; currentSpeakerId?; timeRemaining?; votesCast?; votesTotal?; lastElimination? }
PlayerSecret = { role: Role; wordOrNull: string|null; revealAllowed: boolean }

9.3 State machine (tabla de transiciones)

Estado	Evento	Nuevo estado	Validación / efectos
LOBBY	host:start	REVEAL	Valida conteos = listos; asigna roles/palabras; revealAllowed=true
REVEAL	host:closeReveal	CLUES	Si mode=LIVE => revealAllowed=false; genera turnOrder; inicia timer si aplica
CLUES	timer end / turn:next	CLUES o VOTE	Avanza turnIndex; si termina orden => VOTE
VOTE	auto all votes / vote:close	RESOLVE	Calcula tally; elige eliminado (random si empate)
RESOLVE	post-resolve	CLUES o END	Revela rol; evalúa victoria; si no termina => nueva ronda Clues
END	host:resetRoom	LOBBY	Resetea sala (mantener roomCode o crear nueva según implementación)

9.4 Algoritmos y reglas (MVP)

Asignación de roles

- Crear lista de roles según settings.roleCounts y mezclar (Fisher-Yates).
- Asignar roles a players listos en orden aleatorio.
- Asignar palabra: Civil -> A; Undercover -> B; MrWhite -> null.
- Enviar player:secret de forma privada.

Turnos y timer

- El server es autoritativo: emite turn:update cada 1s con timeRemaining.

- Si turnSeconds=null: el host avanza con turn:next.
- Al terminar el último hablante, pasar automáticamente a VOTE.

Votación

- Permitir votar solo si status=VOTE y voterId está vivo.
- Guardar votes[voterId]=targetId; permitir cambiar hasta cierre.
- Cierre automático cuando votesCast == aliveCount.
- Tally: contar votos por targetId; elegir max; empate: random entre empatados.

Reconexión

- Client genera playerToken (UUID) en localStorage al unirse.
- room:join acepta playerToken; si coincide con jugador existente, re-asociar socketId.
- Re-enviar player:secret (según revealAllowed) al reconectar.

Pantalla pública (/public)

- Se conecta como spectator (sin voto, sin secreto).
- Muestra fase, turno, timer, vivos/eliminados, votos emitidos (conteo), último eliminado y rol.

Manejo de host

- Generar hostKey al crear sala; guardar en localStorage del host.
- Validar hostKey en eventos host:*.
- MVP: si host se desconecta, finalizar sala y notificar.

10. Plan de pruebas y criterios de aceptación

10.1 Criterios de aceptación (MVP)

- EN VIVO: después de cerrar Reveal, ningún jugador puede volver a ver su palabra (ni por UI ni por evento).
- EN VIVO: en reposo, todas las pantallas son neutras y no delatan rol.
- Votación: no se puede votar fuera de VOTE; se respeta una vez al cierre.
- Empates: el sistema elimina al azar entre empatados.
- Reconexión: recargar vuelve a vivo/muerto y a la fase correcta.
- END: se muestran ganador y palabras A/B.

10.2 Tests sugeridos

Unit tests (server)

- Asignación de roles respeta conteos y distribuye palabras correctamente.
- Transiciones válidas/invalidas (no votar en Reveal, etc.).
- Tally + desempate aleatorio (controlando RNG con seed en test).
- Condiciones de victoria para presets SIMPLE y CLASSIC_GUESS.

Integración (socket)

- Flujo completo 6 jugadores: lobby -> reveal -> clues -> vote -> resolve -> end.
- Reconexión durante Clues y durante Vote.
- Pantalla pública no recibe secretos.

11. Deployment gratis en Mac (túneles)

Objetivo: exponer tu server local con HTTPS sin pagar. MVP recomendado: Cloudflare Quick Tunnels (URL efímera).

Opción A — Cloudflare Quick Tunnels (recomendado)

Instalar cloudflared y correr un comando que expone tu localhost:

```
# levantar app en localhost:3000
npm run dev

# abrir túnel
cloudflared tunnel --url http://localhost:3000
```

Notas:

- La URL suele cambiar cada vez (túnel quick / trycloudflare).
- Ideal para jugar con amigos; no es un dominio estable.

Opción B — ngrok (plan free)

Alternativa popular con límites en el plan gratis (ej. 1 GB data transfer out y 20k requests/mes; hasta 3 endpoints).

Opción C — localtunnel

CLI simple vía npm: lt --port 3000. Confiabilidad variable.

Opción D — Tailscale Funnel

Si ya usás Tailscale, Funnel permite exponer un servicio con HTTPS. Requiere setup de tailnet.

Referencias (docs oficiales)

- Cloudflare Quick Tunnels (trycloudflare): https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/do-more-with-tunnels/trycloudflare/
- ngrok Free Plan Limits: https://ngrok.com/docs/pricing-limits/free-plan-limits
- localtunnel (npm): https://www.npmjs.com/package/localtunnel
- Tailscale Funnel: https://tailscale.com/docs/features/tailscale-funnel/

12. Roadmap (futuro): audio/video con LiveKit

No se implementa en MVP. Para una v2 remota, se puede integrar LiveKit para A/V embebido y data channels.

Opciones

- Integración con LiveKit Cloud (rápido pero ya es SaaS).
- Self-host LiveKit (DIY; requiere setup y networking).
- Mantener A/V externo y solo usar canales de datos si aplica.

Referencia

Repositorio open source de LiveKit: https://github.com/livekit/livekit

Apéndice A — Esquema de mazo de palabras (JSON)

Formato recomendado (local en el client):

```
[
  { "category": "Comida", "a": "Pizza", "b": "Empanada" },
  { "category": "Lugares", "a": "Playa", "b": "Desierto" }
]
```

Apéndice B — Reglas de privacidad (recordatorio)

- No almacenar emails, teléfonos ni identificadores reales.
- Borrar salas al resetear o al reiniciar server.
- Evitar logs que incluyan palabras/roles en consola por defecto (permitir debug mode opcional).